

Lecture 12: Conjugate Direction and Conjugate Gradient Methods

Instructor Proof Notes

Color Legend

Green: Step numbering and substeps **Orange:** Key concepts **Purple:** Main results **Blue:** Teaching notes

Proof Roadmap

Theorem	Time	Board-Friendly	Priority
Theorem 1: Q-Conjugacy $\Rightarrow$ Linear Independence	5–8 min	Yes	Full
Theorem 2: Gradient Orthogonality Property	10–12 min	Yes	Full
Theorem 3: Expanding Subspace Theorem	12–15 min	Moderate	Full
Theorem 4: Convergence in n Steps	3–5 min	Yes	Full
Theorem 5: CG Produces Q-Conjugate Directions	8–10 min	Sketch	Key Result

1 Theorem 1: Q-Conjugate Directions Are Linearly Independent

Statement: Let Q be a symmetric positive definite (SPD) matrix. If $d_1, d_2, \dots, d_k \in \mathbb{R}^n$ are **Q-conjugate directions**, meaning $d_i^T Q d_j = 0$ for all $i \neq j$, then they are **linearly independent**.

Proof:

Step 1: Suppose the directions are linearly dependent. Then there exist scalars $\alpha_1, \alpha_2, \dots, \alpha_k$ (not all zero) such that

$$\alpha_1 d_1 + \alpha_2 d_2 + \dots + \alpha_k d_k = 0.$$

Step 2: Multiply both sides on the left by $d_i^T Q$ for any fixed $i \in \{1, \dots, k\}$:

$$d_i^T Q (\alpha_1 d_1 + \alpha_2 d_2 + \dots + \alpha_k d_k) = d_i^T Q \cdot 0 = 0.$$

Step 3: Expand the left side:

$$\alpha_1 (d_i^T Q d_1) + \alpha_2 (d_i^T Q d_2) + \dots + \alpha_i (d_i^T Q d_i) + \dots + \alpha_k (d_i^T Q d_k) = 0.$$

Step 4: By Q-conjugacy, $d_i^T Q d_j = 0$ for all $j \neq i$. Thus all cross terms vanish, leaving

$$\alpha_i (d_i^T Q d_i) = 0.$$

Step 5: Since Q is SPD and $d_i \neq 0$ (all directions are nonzero), we have $d_i^T Q d_i > 0$. Therefore,

$$\alpha_i = 0.$$

Step 6: Since i was arbitrary, $\alpha_1 = \alpha_2 = \dots = \alpha_k = 0$, contradicting the assumption of linear dependence.

The directions $d_1, \dots, d_k$ are linearly independent.

Teaching Notes: Theorem 1

Board Strategy: This is a short, clean proof — excellent for chalk talk. Draw an analogy with ordinary orthogonality first.

Key Insight: Q -conjugacy is the “ Q -inner product” analogy to orthogonality. Just as orthogonal vectors are linearly independent, so are Q -conjugate vectors.

Diagram: Draw 2D example: two orthogonal vectors v_1, v_2 are linearly independent. Then explain that Q -conjugacy replaces the standard dot product with the Q -weighted inner product $d_i^T Q d_j$.

Timing: 5–8 minutes, including the analogy. This proof can be rushed if time is tight.

Common Student Confusion

Q: “Why can’t we just use orthogonal directions instead of Q -conjugate?”

A: Because the underlying optimization problem is weighted by Q . The function $f(x) = \frac{1}{2}x^T Q x + c^T x$ is naturally expressed in the Q -inner product. Directions that are orthogonal in the standard sense may not be “aligned” with the contours of f . Q -conjugacy ensures that once we optimize along d_i , we never need to revisit that direction again.

2 Theorem 2: Gradient Orthogonality Property

Statement: In the **conjugate direction method**, suppose we minimize $f(x) = \frac{1}{2}x^T Q x + c^T x$ starting from x_1 with directions $d_1, \dots, d_n$ that are Q -conjugate. At iteration k , denote $\nabla_k = \nabla f(x_k)$. Then,

$$\nabla_{k+1}^T d_i = 0 \quad \text{for all } i = 1, 2, \dots, k.$$

Proof (by induction on k):

Base case ($k = 1$): **Step 1:** At iteration 1, we compute $x_2 = x_1 + \lambda_1 d_1$ where λ_1 minimizes $f(x_1 + \lambda d_1)$ (exact line search). **Step 2:** By first-order optimality of the line search,

$$\left. \frac{d}{d\lambda} f(x_1 + \lambda d_1) \right|_{\lambda=\lambda_1} = 0.$$

Step 3: Compute the derivative:

$$\frac{d}{d\lambda} f(x_1 + \lambda d_1) = \nabla f(x_1 + \lambda d_1)^T d_1 = (Q(x_1 + \lambda d_1) + c)^T d_1.$$

Step 4: At $\lambda = \lambda_1$, this is $\nabla_2^T d_1 = 0$. ✓

Inductive step: Assume $\nabla_k^T d_i = 0$ for all $i = 1, \dots, k-1$. Show $\nabla_{k+1}^T d_i = 0$ for all $i = 1, \dots, k$.

Step 5: For $i = k$, the base case argument applies: $\nabla_{k+1}^T d_k = 0$ from the line search at iteration k .

Step 6: For $i < k$, we have

$$\nabla_{k+1}^T d_i = (Qx_{k+1} + c)^T d_i \quad (1)$$

$$= (Q(x_k + \lambda_k d_k) + c)^T d_i \quad (2)$$

$$= (Qx_k + c)^T d_i + \lambda_k (Qd_k)^T d_i \quad (3)$$

$$= \nabla_k^T d_i + \lambda_k d_k^T Q d_i. \quad (4)$$

Step 7: By the induction hypothesis, $\nabla_k^T d_i = 0$. By Q-conjugacy, $d_k^T Q d_i = 0$ (since $i < k$). Thus,

$$\nabla_{k+1}^T d_i = 0 + 0 = 0. \quad \checkmark$$

By induction, $\nabla_{k+1}^T d_i = 0$ for all $i = 1, \dots, k$.

Teaching Notes: Theorem 2

Board Strategy: Write the induction clearly; this is the backbone of the whole theory.

Key Insight: The magic is in Step 7: two terms cancel separately, using *both* the induction hypothesis AND the Q-conjugacy property. Students often miss that both are needed.

Intuition: After we move along d_k , the gradient is orthogonal (in the Q-sense) to all previous directions. We never need to revisit them.

Timing: 10–12 minutes, including setup. This proof is worth the time—it justifies the whole algorithm.

Optional: Draw a 2D picture showing $\nabla_1, \nabla_2, \nabla_3$ and how they point in different “Q-orthogonal” directions.

Common Student Confusion

Q: “I thought the gradient should go to zero at the minimum. How can $\nabla_{k+1}^T d_i = 0$ before we’re done?”

A: The gradient ∇_{k+1} is not zero; rather, it is orthogonal (in the Q-sense) to the subspace spanned by $d_1, \dots, d_k$. The gradient still has a component in the direction d_{k+1} . Once we include all n conjugate directions and minimize in each, the gradient will have nowhere else to point, and it becomes zero.

3 Theorem 3: Expanding Subspace Theorem

Statement: After k iterations of the conjugate direction method, the current point x_{k+1} is the **global minimizer of f over the affine subspace $\{x_1\} + \text{span}(d_1, \dots, d_k)$** .

Proof:

Step 1: Define the **reduced problem**: for a displacement vector $\alpha \in \mathbb{R}^k$, let

$$\theta_k(\alpha) = f(x_1 + D^k \alpha),$$

where $D^k = [d_1 \ d_2 \ \cdots \ d_k] \in \mathbb{R}^{n \times k}$ is the matrix with columns $d_1, \dots, d_k$.

Step 2: Compute the gradient of θ_k :

$$\nabla \theta_k(\alpha) = (D^k)^T \nabla f(x_1 + D^k \alpha) = (D^k)^T \nabla(x_1 + D^k \alpha).$$

Step 3: At $\alpha = \lambda^k := [\lambda_1, \lambda_2, \dots, \lambda_k]^T$ (the step sizes taken through iteration k), we have $x_1 + D^k \lambda^k = x_{k+1}$. Thus,

$$\nabla \theta_k(\lambda^k) = (D^k)^T \nabla_{k+1}.$$

Step 4: By Theorem 2, for each $i = 1, \dots, k$, we have $\nabla_{k+1}^T d_i = 0$. Therefore,

$$\nabla \theta_k(\lambda^k) = (D^k)^T \nabla_{k+1} = \begin{pmatrix} d_1^T \nabla_{k+1} \\ \vdots \\ d_k^T \nabla_{k+1} \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix} = \mathbf{0}.$$

Step 5: The Hessian of θ_k is $(D^k)^T Q D^k$, which is SPD because D^k has full column rank (Theorem 1) and Q is SPD.

Step 6: Since θ_k is a convex quadratic function with SPD Hessian, λ^k is the **unique global minimizer** of θ_k .

The point x_{k+1} globally minimizes f over $\{x_1\} + \text{span}(d_1, \dots, d_k)$.

Teaching Notes: Theorem 3

Board Strategy: The reduced-problem view is abstract but powerful. Sketch a picture: Start at x_1 . After 1 iteration, we've minimized along the line through d_1 . After 2 iterations, we've minimized over the plane spanned by d_1, d_2 . And so on.

Key Insight: This theorem is the geometric heart of conjugate directions. Each iteration expands the subspace over which we've optimized, and we never backtrack.

SPD Hessian: Emphasize that $(D^k)^T Q D^k$ is SPD because (a) the columns of D^k are linearly independent and (b) Q is SPD. This guarantees a unique minimizer.

Timing: 12–15 minutes. This is moderately complex; don't rush. The reduced-problem formulation is worth dwelling on.

Common Student Confusion

Q: “How do we know that minimizing over $\{x_1\} + \text{span}(d_1, \dots, d_k)$ is progress toward the global minimum?”

A: Because $\{x_1\} + \text{span}(d_1, \dots, d_k)$ grows with k . After n iterations, this subspace is all of $\mathbb{R}^n$ (since $d_1, \dots, d_n$ span $\mathbb{R}^n$), so we've minimized over everything.

4 Theorem 4: Convergence in n Steps

Statement: The conjugate direction method converges to the **global minimizer** of $f(x) = \frac{1}{2}x^T Qx + c^T x$ (with Q SPD) in **at most n steps**.

Proof:

Step 1: By Theorem 1, the Q -conjugate directions $d_1, \dots, d_n$ are linearly independent and thus span $\mathbb{R}^n$.

Step 2: By Theorem 3, after k iterations, x_{k+1} minimizes f over $\{x_1\} + \text{span}(d_1, \dots, d_k)$.

Step 3: When $k = n$, the subspace $\{x_1\} + \text{span}(d_1, \dots, d_n) = \mathbb{R}^n$ (the entire space).

Step 4: Therefore, x_{n+1} minimizes f over all of $\mathbb{R}^n$.

Step 5: Since f is strictly convex (Hessian Q is SPD), this global minimum is unique and satisfies $\nabla f(x_{n+1}) = 0$.

The conjugate direction method converges to the global minimizer in at most n iterations.

Teaching Notes: Theorem 4

Board Strategy: This is the payoff theorem—students have been waiting for it. Keep it snappy (3–5 min); it’s a simple corollary of Theorems 1–3.

Key Insight: Conjugate Direction methods are *finite-terminating*. Unlike gradient descent (which can take infinitely many steps), we are guaranteed to solve the problem in exactly n iterations (in exact arithmetic).

Caveat: This finite-termination property holds *exactly* only for quadratic functions. For general non-linear functions, conjugate gradient becomes an iterative method.

Timing: 3–5 minutes. Can be delivered quickly after Theorems 1–3.

Common Student Confusion

Q: “Does this mean conjugate gradient is better than gradient descent?”

A: For *quadratic* functions, yes—finite termination is definitive. For general functions, both are iterative; CG often converges faster in practice due to better direction choices, but we lose the finite-termination guarantee. The quadratic case is where CG shines theoretically.

5 Theorem 5: CG Produces Q-Conjugate Directions

Statement: The **Conjugate Gradient (CG) method** is defined by

$$d_1 = -\nabla_1, \quad (5)$$

$$d_{k+1} = -\nabla_{k+1} + \beta_k d_k, \quad \text{where} \quad \beta_k = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}, \quad (6)$$

with $x_{k+1} = x_k + \lambda_k d_k$ where λ_k is chosen by exact line search. The directions $d_1, d_2, \dots$ are **Q-conjugate**, i.e., $d_i^T Q d_j = 0$ for all $i \neq j$.

Proof Sketch:

Step 1: We prove by induction that $d_k^T Q d_i = 0$ for all $i < k$.

Step 2: **Base case** ($k = 2$): We have $d_2 = -\nabla_2 + \beta_1 d_1$ with $\beta_1 = \frac{\nabla_2^T Q d_1}{d_1^T Q d_1}$.

Step 3: Compute $d_2^T Q d_1$:

$$d_2^T Q d_1 = (-\nabla_2 + \beta_1 d_1)^T Q d_1 = -\nabla_2^T Q d_1 + \beta_1 d_1^T Q d_1.$$

Step 4: Substitute β_1 :

$$d_2^T Q d_1 = -\nabla_2^T Q d_1 + \frac{\nabla_2^T Q d_1}{d_1^T Q d_1} \cdot d_1^T Q d_1 = -\nabla_2^T Q d_1 + \nabla_2^T Q d_1 = 0. \quad \checkmark$$

Step 5: **Inductive step:** Assume $d_i^T Q d_j = 0$ for all $j < i < k$. We must show $d_{k+1}^T Q d_i = 0$ for all $i \leq k$.

Step 6: For $i = k$: By the same calculation as the base case (with β_k chosen precisely to ensure this), we have $d_{k+1}^T Q d_k = 0$.

Step 7: For $i < k$: We use the recurrence $d_{k+1} = -\nabla_{k+1} + \beta_k d_k$ and the relation

$$d_{k+1}^T Q d_i = -\nabla_{k+1}^T Q d_i + \beta_k d_k^T Q d_i.$$

Step 8: By Theorem 2, $\nabla_{k+1}^T d_i = 0$. Differentiating the line search condition and using properties of the algorithm, one can show $\nabla_{k+1}^T Q d_i = 0$ for $i < k$ as well. By the induction hypothesis, $d_k^T Q d_i = 0$. Thus, $d_{k+1}^T Q d_i = 0$.

By induction, all CG directions are Q-conjugate.

Teaching Notes: Theorem 5

Board Strategy: The full rigorous proof is technical; instead, **sketch the base case** (the derivation of β_1) and state the inductive step.

Key Insight: The magic of CG is the choice of β_k . By picking $\beta_k = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}$, we ensure that d_{k+1} is Q-conjugate to d_k *using only the previous direction*, not all previous directions. This is why CG is so efficient—linear storage.

Reference: For the full proof, cite Nocedal & Wright, Theorem 5.3, or a standard reference on nonlinear optimization.

Diagram Idea: Draw a 2D example showing ∇_1, d_1 , then $\nabla_2, d_2 = -\nabla_2 + \beta_1 d_1$, showing geometrically how $\beta_1 d_1$ “bends” $-\nabla_2$ to be Q-conjugate to d_1 .

Timing: 8–10 minutes for sketch + intuition. Full proof can be assigned as reading.

Common Student Confusion

Q: “Why does CG only use one previous direction instead of all of them? Won’t we lose the Q-conjugacy property?”

A: This is a beautiful fact: by a nontrivial argument (involving Theorem 2 and properties of quadratic functions), Q-conjugacy with previous *all* directions is automatically guaranteed if you maintain Q-conjugacy with the immediately previous direction. The induction proof shows why. This is why CG is so efficient—we only store d_k and don’t need to store $d_1, \dots, d_{k-1}$.

Summary and Connections

The Big Picture:

1. **Theorem 1** shows that Q-conjugate directions cannot be redundant.
2. **Theorem 2** shows that the gradient becomes orthogonal to past directions—we never need to revisit them.
3. **Theorem 3** formalizes this: each iteration expands the subspace we’ve optimized over.
4. **Theorem 4** concludes: we solve the problem in exactly n steps.
5. **Theorem 5** provides a practical algorithm (CG) that automatically produces Q-conjugate directions using a simple recursive formula.

Why This Matters: These theorems transform the problem from “how do we find a good direction?” to “how do we find conjugate directions?” The CG answer is elegant: use a one-parameter update rule, and conjugacy is maintained automatically.

Practical Implementation Notes:

- In practice, numerical round-off errors accumulate. After n steps, x_n may not be exactly optimal; restarting or using a tolerance is common.
- For non-quadratic functions, CG is restarted every n iterations or when some condition (e.g., $\|\nabla_k\|^2$ increase) is detected.
- The formula $\beta_k = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}$ requires computing $Q d_k$, which is one matrix-vector product per iteration.

Recommended Homework / Discussion

- Verify Theorem 1 in a 2D example with a non-identity Q .
- Implement CG for a small $n \times n$ problem and observe exact convergence in n steps.
- Derive the Polak-Ribière formula as an alternative expression for β_k .
- Discuss why CG fails on non-convex problems (loss of finite termination guarantee).